

TASKFLOW

Task Management System

Project Report

Developed Using

React.js • Vite • Tailwind CSS • FastAPI • SQLAlchemy • SQLite • JWT

Submitted By

Yash Jobanputra

MSc ICT

Veer Narmad South Gujarat University (VNSGU)

2026

1. Introduction

TaskFlow is a full-stack Task Management System developed to provide a simple and secure way for users to manage personal tasks. The application supports user registration, login, JWT-based authentication, task creation, viewing, editing, completion, deletion and search.

2. Objectives

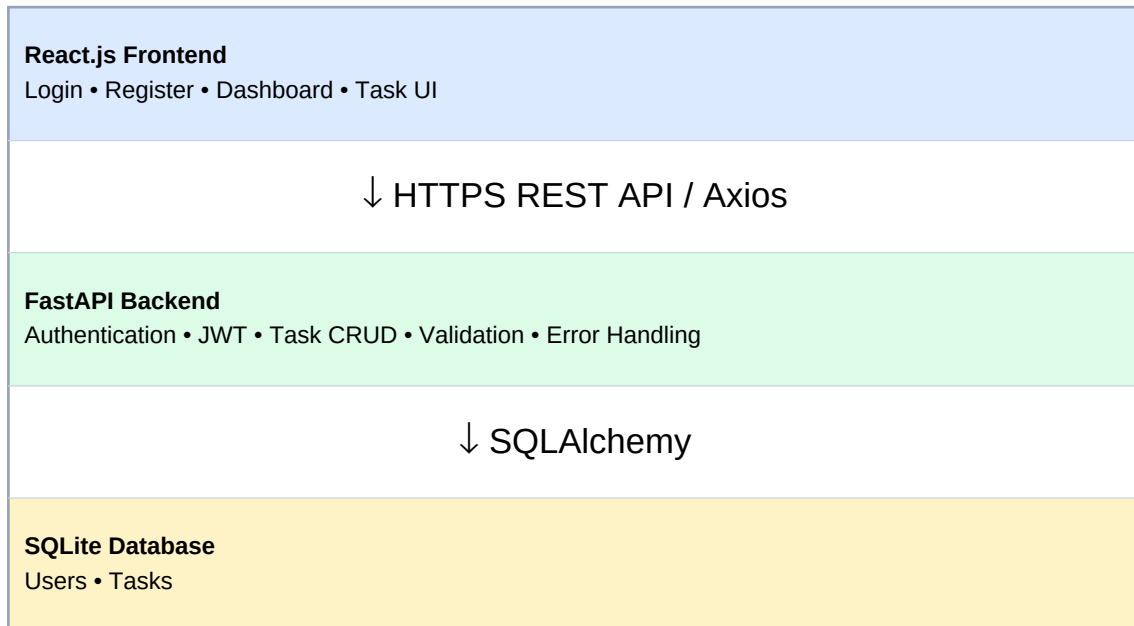
The main objectives of the project are to build a practical REST API, implement secure user authentication, provide complete CRUD operations, validate user input, handle application errors and deploy the complete application so it can be accessed online.

3. Technologies Used

Layer	Technology
Frontend	React.js, Vite, Tailwind CSS, Axios, React Router
Backend	Python, FastAPI
ORM / Database Access	SQLAlchemy
Database	SQLite
Authentication	JWT, Passlib, Bcrypt
API Documentation	FastAPI Swagger / OpenAPI
Version Control	Git, GitHub
Deployment	Render

4. System Architecture

TaskFlow follows a frontend-backend architecture. The React frontend communicates with the FastAPI backend through REST APIs. The backend authenticates users using JWT tokens and uses SQLAlchemy to communicate with the SQLite database.



5. Main Modules

User Authentication: Users can register with username, email and password and can login to receive a JWT access token.

Task Management: Authenticated users can create, read, update and delete their own tasks.

Task Status: Tasks can be marked as pending or completed.

Search: Users can search their task list by title.

Error Handling: The frontend displays useful error messages and redirects unauthorized users to the login page.

6. Authentication and Authorization

TaskFlow uses JWT (JSON Web Token) authentication. During login, the backend verifies the supplied username and password. When authentication succeeds, the API returns an access token. The frontend stores the token and sends it in the Authorization header for protected task requests.

Authorization Header

Authorization: Bearer <access_token>

Task APIs identify the current user from the JWT token and restrict task operations to that user's records.

7. REST API Endpoints

Method	Endpoint	Purpose
POST	/auth/register	Register a new user
POST	/auth/login	Authenticate a user
POST	/tasks/	Create a task
GET	/tasks/	Get user's tasks
GET	/tasks/{task_id}	Get one task
PUT	/tasks/{task_id}	Update a task
DELETE	/tasks/{task_id}	Delete a task

Interactive API documentation is available through FastAPI Swagger at:
<https://taskflow-api-ty4q.onrender.com/docs>

8. Database Design

The application uses SQLite as its database and SQLAlchemy as the ORM. The main entities are User and Task. A user can own multiple tasks, while each task belongs to one user.

Entity	Important Fields	Relationship
User	id, username, email, password	One User → Many Tasks
Task	id, title, description, completed, user_id	Many Tasks → One User

9. Validation and Error Handling

The application validates registration and task input, handles invalid credentials, protects task endpoints from unauthorized access, returns 404 responses when a task is not found and provides user-friendly frontend error messages.

10. Deployment

The frontend and backend are deployed online using Render. The frontend communicates with the deployed FastAPI backend through HTTPS.

Live Frontend: <https://taskflow-10g1.onrender.com>

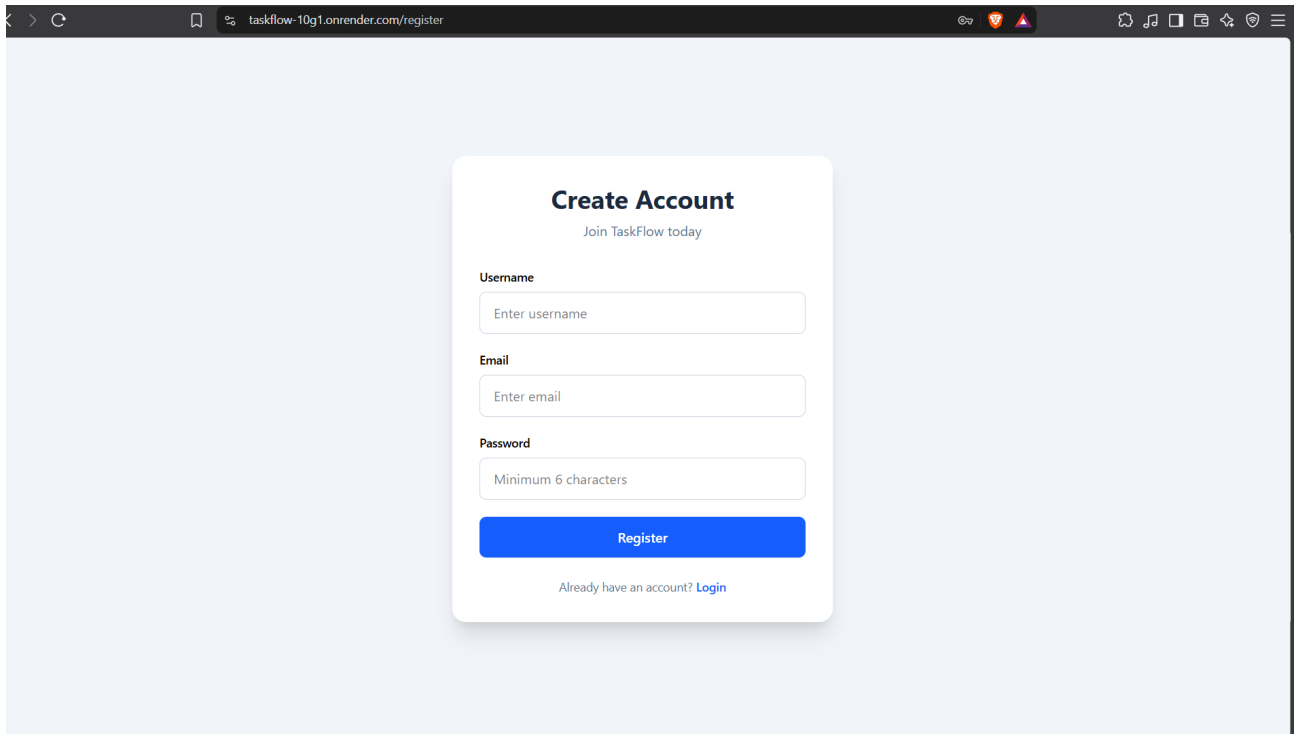
Backend: <https://taskflow-api-ty4q.onrender.com>

API Documentation: <https://taskflow-api-ty4q.onrender.com/docs>

GitHub: <https://github.com/jobanputrayash22-cpu/TaskFlow>

11. Screenshots

The following screenshots demonstrate the main functionality of the deployed TaskFlow application.



The screenshot shows a web browser window with the URL `taskflow-10g1.onrender.com/register`. The page features a central white card with a light blue background. The card is titled "Create Account" in bold black text, with the subtitle "Join TaskFlow today" below it. The registration form includes three input fields: "Username" with the placeholder "Enter username", "Email" with the placeholder "Enter email", and "Password" with the placeholder "Minimum 6 characters". A blue "Register" button is positioned below the password field. At the bottom of the card, there is a link that says "Already have an account? [Login](#)".

Figure 1: User Registration Page

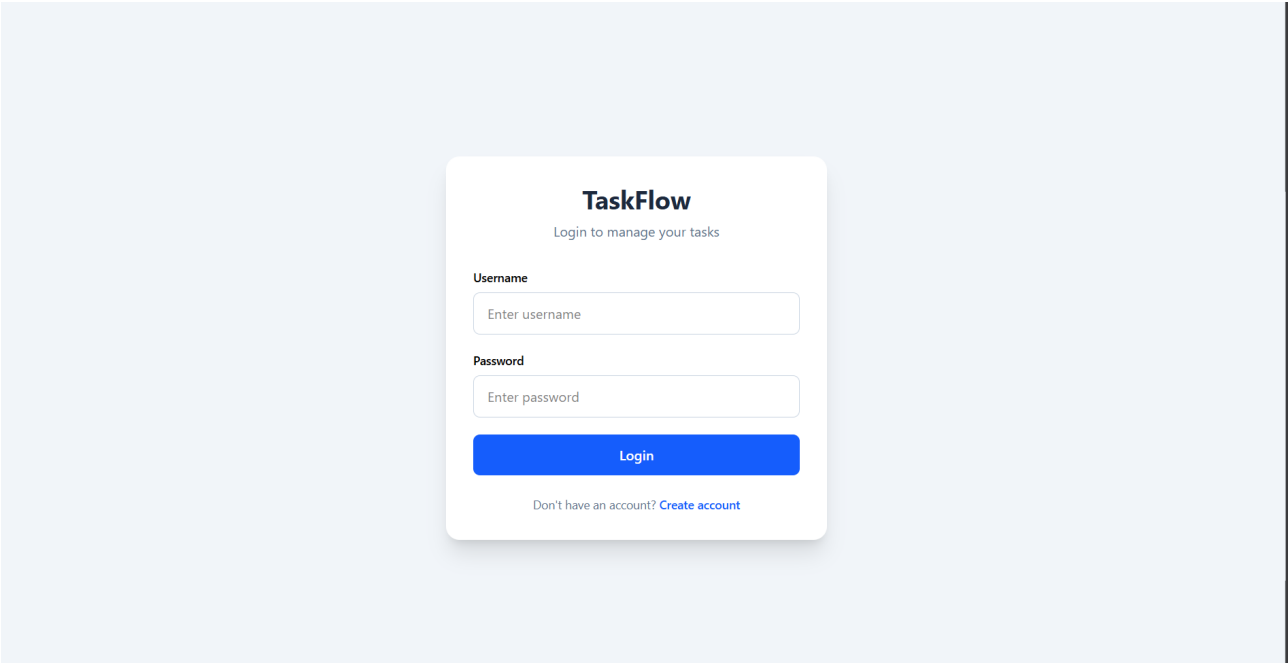


Figure 2: User Login Page

Task Dashboard

Add New Task

No tasks found

Figure 3: Task Dashboard

Task Dashboard

Add New Task

Add TaskSearchAll

No tasks found

Figure 4: Adding a New Task

Task Dashboard

Edit Task

Update Task

Cancel

Search

All

~~movie watch~~

kgf movie watch tonight

Completed

✓

Edit

Delete

Figure 5: Editing an Existing Task

Task Dashboard

Add New Task

movie watch

kgf movie watch tonight

Completed

Figure 6: Completed Task

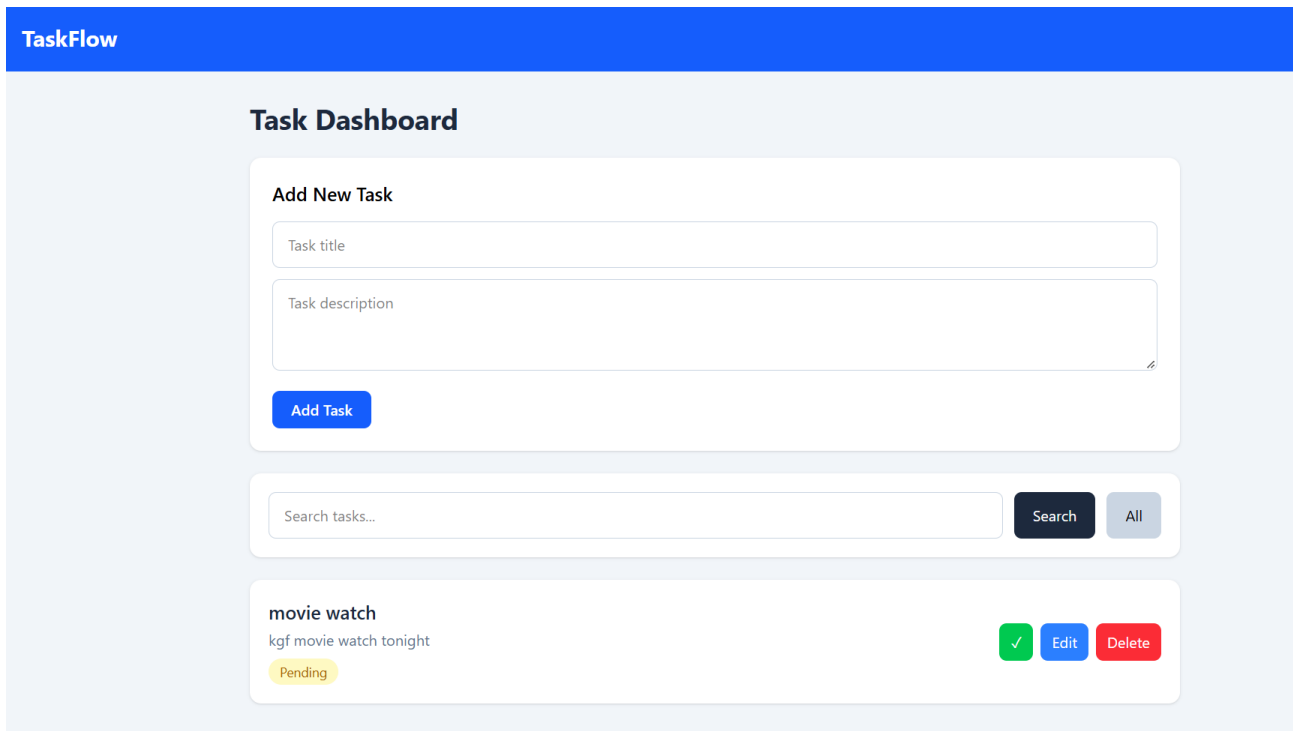


Figure 7: Delete Task Confirmation

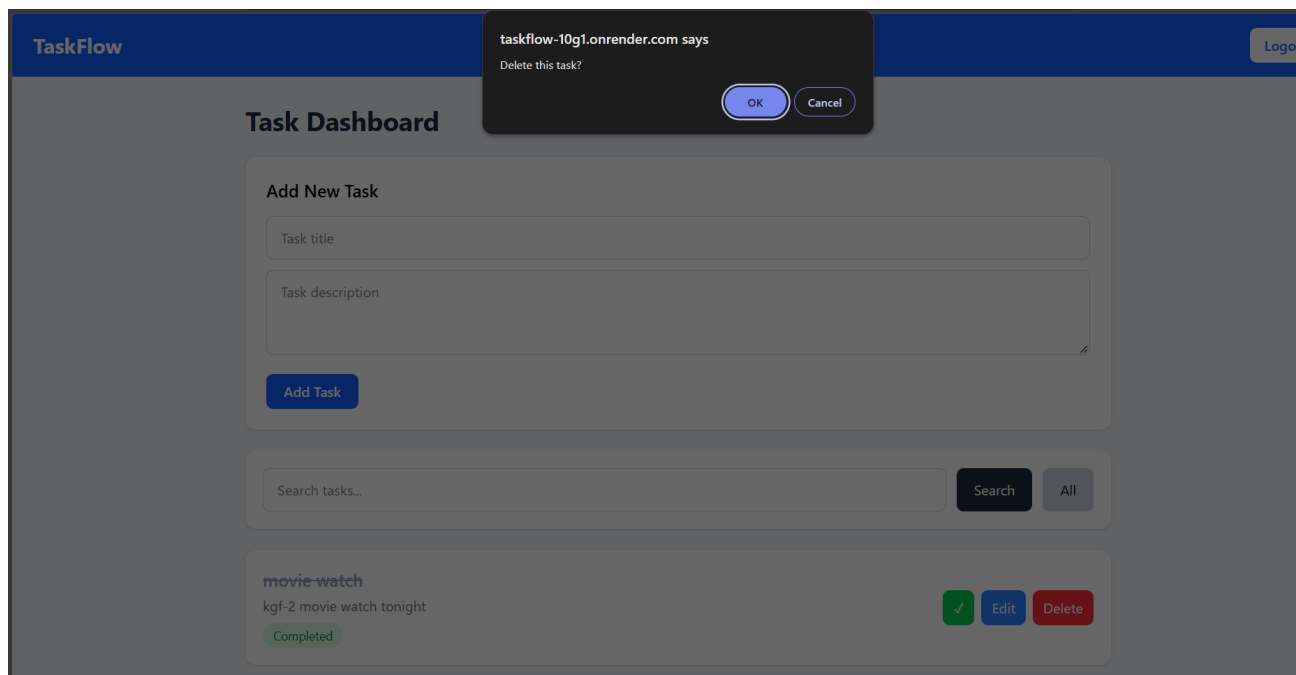


Figure 8: Task Dashboard After Task Operations

12. Testing

Test Case	Expected Result	Status
User Registration	New account is created	Passed
User Login	JWT access token is returned	Passed
Create Task	Task is added to user's list	Passed
View Tasks	User's tasks are displayed	Passed
Edit Task	Task details are updated	Passed
Complete Task	Task status changes	Passed
Search Task	Matching tasks are displayed	Passed
Delete Task	Selected task is removed	Passed
Unauthorized Access	User is redirected to login	Passed

13. Future Enhancements

Possible future improvements include task due dates, priorities, categories, pagination, advanced filtering, email notifications, a PostgreSQL database, user profile management and an admin dashboard.

14. Conclusion

TaskFlow successfully demonstrates a complete full-stack task management application. It combines a React-based user interface with a FastAPI REST backend, JWT authentication, SQLAlchemy database integration and complete task CRUD functionality. The deployed application provides a practical demonstration of authentication, authorization, API development, validation, error handling and deployment.

End of Report